arXiv:1802.05779v2 [quant-ph] 12 Jan 2019

Quantum Variational Autoencoder

Amir Khoshaman*,[1] Walter Vinci*,[1] Brandon Denis,[1] Evgeny Andriyash,[1] Hossein Sadeghi,[1] and Mohammad H. Amin[1,2]

[1]*D-Wave Systems Inc., 3033 Beta Avenue, Burnaby BC Canada V5G 4M9*
[2]*Department of Physics, Simon Fraser University, Burnaby, BC Canada V5A 1S6*

Variational autoencoders (VAEs) are powerful generative models with the salient ability to perform inference. Here, we introduce a *quantum variational autoencoder* (QVAE): a VAE whose latent generative process is implemented as a quantum Boltzmann machine (QBM). We show that our model can be trained end-to-end by maximizing a well-defined loss-function: a "quantum" lower-bound to a variational approximation of the log-likelihood. We use quantum Monte Carlo (QMC) simulations to train and evaluate the performance of QVAEs. To achieve the best performance, we first create a VAE platform with discrete latent space generated by a restricted Boltzmann machine (RBM). Our model achieves state-of-the-art performance on the MNIST dataset when compared against similar approaches that only involve discrete variables in the generative process. We consider QVAEs with a smaller number of latent units to be able to perform QMC simulations, which are computationally expensive. We show that QVAEs can be trained effectively in regimes where quantum effects are relevant despite training via the quantum bound. Our findings open the way to the use of quantum computers to train QVAEs to achieve competitive performance for generative models. Placing a QBM in the latent space of a VAE leverages the full potential of current and next-generation quantum computers as sampling devices.

I. INTRODUCTION

While rooted in fundamental ideas that date back decades ago [1, 2], deep-learning algorithms [3, 4] have only recently started to revolutionize the way information is collected, analyzed, and interpreted in almost every intellectual endeavor [5]. This is made possible by the computational power of modern dedicated processing units (such as GPUs). The most remarkable progress has been made in the field of supervised learning [6], which requires a labeled dataset. There has also been a surge of interest in the development of unsupervised learning with unlabeled data [4, 7, 8]. One notable challenge in unsupervised learning is the computational complexity of training most models [9].

It is reasonable to hope that some of the computational tasks required to perform both supervised and unsupervised learning could be significantly accelerated by the use of *quantum processing units* (QPU). Indeed, there are already quantum algorithms that can accelerate machine learning tasks [10–13]. Interestingly, machine learning algorithms have been used in quantum-control techniques to improve fidelity and coherence [14–16]. This natural interplay between machine learning and quantum computation is stimulating a rapid growth of a new research field known as *quantum machine learning* [17–21].

A full implementation of quantum machine-learning algorithms requires the construction of fault-tolerant QPUs, which is still challenging [22–24]. However, the remarkable recent development of gate-model processors with a few dozen qubits [25, 26] and quantum annealers with a few thousand qubits [27, 28] has triggered an interest in developing quantum machine-learning algorithms that can be practically tested on current and near-future quantum devices. Early attempts to use small gate-model devices for machine learning use techniques similar to those developed in the context of quantum approximate optimization algorithms (QAOA) [29] and variational quantum algorithms (VQA) [26, 30] to perform quantum heuristic optimization as a subroutine for small unsupervised tasks such as clustering [31]. The use of quantum annealing devices for machine-learning tasks is perhaps more established and relies on the ability of quantum annealers to perform both optimization [32–34] and sampling [35–38].

As optimizers, quantum annealers have been used to perform supervised tasks such as classification [39–42]. As samplers, they have been used to train RBMs, and are thus well-suited to perform unsupervised tasks such as training deep probabilistic models [43–46]. In Ref. [47], a D-Wave quantum annealer was used to train a deep network of stacked RBMs to classify a coarse-grained version of the MNIST dataset [48]. Quantum annealers have also been used to train fully visible Boltzmann machines on small synthetic datasets [49, 50]. While mostly used in conjunction with traditional RBMs, quantum annealing should find a more natural application in the training of QBM [51].

A clear disadvantage of such early approaches is the need to consider datasets with a small number of input units, which prevents a clear route towards practical applications of quantum annealing with current and next-generation devices. A first attempt towards this end was presented in Ref. [52], with the introduction of a quantum-assisted Helmholtz machine (QAHM). However, training QAHM is based on the *wake-sleep* algorithm [8], which does not have a well-defined loss function in the wake and sleep phases of training. Moreover,

*Both authors contributed equally to this work.

the gradients do not correctly propagate in the networks between the two phases. Because of these shortcomings, QAHM generates blurry images and training does not scale to standard machine-learning datasets such as MNIST.

Our approach is to use variational auto-encoders (VAEs), a class of generative models that provide an efficient inference mechanism [53, 54]. We show how to implement a quantum VAE (QVAE), *i.e.*, a VAE with discrete variables (DVAE) [55] whose generative process is realized by a QBM. QBMs were introduced in Ref. [51], and can be trained by minimizing a quantum lower bound to the true log-likelihood. We show that QVAEs can be effectively trained by sampling from the QBM with continuous-time quantum Monte Carlo (CT-QMC). We demonstrate that QVAEs have performance on par with conventional DVAEs equipped with traditional RBMs, despite being trained via an additional bound to the likelihood.

QVAEs share some similarities with QAHMs, such as the presence of both an *inference (encoder)* and a *generation (decoder)* network. However, they have the advantage of a well-defined loss function with fully propagating gradients that can be efficiently trained via backpropagation. This allows to achieve state-of-the-art performance (for models with only discrete units) on standard datasets such as MNIST by training (classical) DVAEs with large RBMs. Training QVAEs with a large number of latent units is impractical with CT-QMC, but can be be accelerated with quantum annealers. Our work thus opens a path to practical machine learning applications with current and next-generation quantum annealers.

The QVAEs we introduce in this work are generative models with a classical autoencoding structure and a quantum generative process. This is in contrast with the quantum autoencoders (QAEs) introduced in Refs. [56, 57]. QAEs have a quantum autoencoding structure (realized via quantum circuits), and can be used for quantum and classical data compression, but lack a generative structure.

The structure of the paper is as follows. In Sec. II we provide a general discussion of generative models with latent variables, which include VAEs as a special case. We then introduce the basics of VAEs with continuous latent variables in Sec. III. Sec. IV discusses the generalization of VAEs to discrete latent variables and presents our experimental results with RBMs implemented in the latent space. In Sec. V we introduce QVAEs and present our results. We conclude in Sec. VI and give further technical and methodological details in the Appendices.

II. GENERATIVE MODELS WITH LATENT VARIABLES

Let $X = \{\mathbf{x}^d\}_{d=1}^N$ represents a training set of N independent and identically distributed samples coming from an unknown data distribution, $p_{\text{data}}(X)$ (for instance, the distribution of the pixels of a set of images). Generative models are probabilistic models that minimize the "distance" between the model distribution, $p_{\boldsymbol{\theta}}(X)$, and the data distribution, $p_{\text{data}}(X)$, where $\boldsymbol{\theta}$ denotes the parameters of the model. Generative models can be categorized in several ways, but for the purpose of this paper we focus on the distinction between models with latent (unobserved) variables and fully-visible models with no latent variables. Examples of the former include generative adversarial networks (GAN) [58], VAEs [53], and RBMs, whereas some important examples of the latter include NADE [59], MADE [60], pixelRNNs, and pixelCNNs [61].

The conditional relationships among the visible units, $\mathbf{x}$, and latent units, $\boldsymbol{\zeta}$, determine the joint probability distribution, $p_{\boldsymbol{\theta}}(\mathbf{x}, \boldsymbol{\zeta})$, of a generative model and can be represented in terms of either *undirected* (Fig. 1(a)) or *directed* (Fig. 1(b)) graphs. Unlike fully visible models, generative models with latent variables can potentially learn and encode in the latent space useful representations of the data. This is an appealing property that can be exploited to improve other tasks such as supervised and semi-supervised learning (*i.e.*, when only a fraction of the input data is labeled) [62] with substantial practicality in image search [63], speech analysis [64], genomics [65], drug design [66], and so on.

Training a generative model is commonly done via maximum likelihood (ML) approach, in which optimal model parameters $\boldsymbol{\theta}^*$ are obtained by maximizing the likelihood of the dataset:

$$\sum_{\mathbf{x}} p_{\text{data}}(\mathbf{x}) \log p_{\boldsymbol{\theta}}(\mathbf{x}) = \mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}[\log p_{\boldsymbol{\theta}}(\mathbf{x})] \,, \tag{1}$$

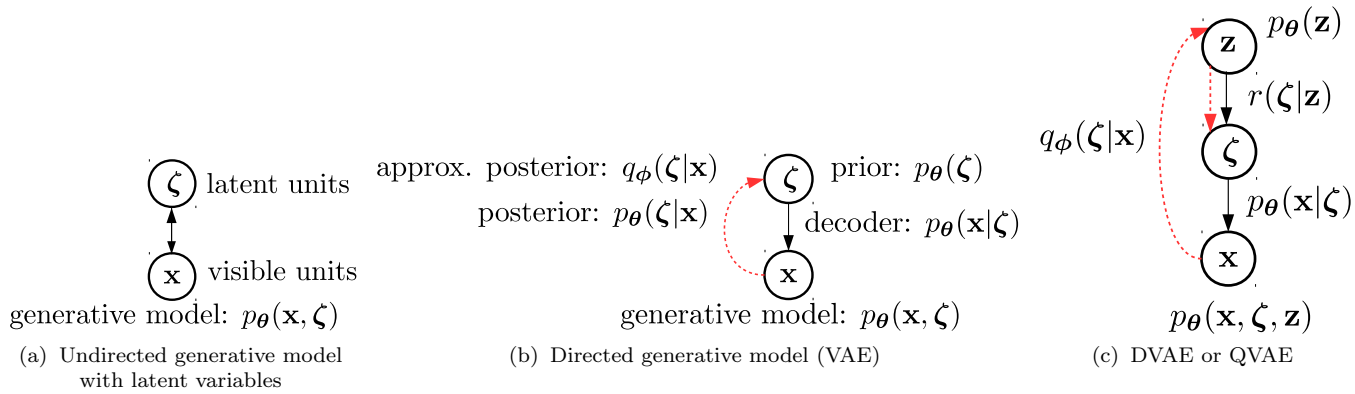
where $p_{\boldsymbol{\theta}}(\mathbf{x}) = \sum_{\boldsymbol{\zeta}} p_{\boldsymbol{\theta}}(\mathbf{x}, \boldsymbol{\zeta})$ is the marginal probability distribution of the visible units and $\mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}[\dots]$ means the expectation value over $\mathbf{x}$ sampled from $p_{\text{data}}(\mathbf{x})$.

To better understand the behavior of generative models with latent variables, we now write $\mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}[\log p_{\boldsymbol{\theta}}(\mathbf{x})]$ in a more insightful form. First, note that $\log p_{\boldsymbol{\theta}}(\mathbf{x}) = \mathbb{E}_{\boldsymbol{\zeta}\sim p_{\boldsymbol{\theta}}(\boldsymbol{\zeta}|\mathbf{x})}[\log p_{\boldsymbol{\theta}}(\mathbf{x})]$, since $p_{\boldsymbol{\theta}}(\mathbf{x})$ is independent of $\boldsymbol{\zeta}$. The quantity $p_{\boldsymbol{\theta}}(\boldsymbol{\zeta}|\mathbf{x})$ is called the *posterior* distribution, since it represents the probability of the latent variables after an observation $\mathbf{x}$ has been made (see Fig. 1(b)). Also, since we have $p_{\boldsymbol{\theta}}(\mathbf{x}) = p_{\boldsymbol{\theta}}(\mathbf{x}, \boldsymbol{\zeta})/p_{\boldsymbol{\theta}}(\boldsymbol{\zeta}|\mathbf{x})$, we can write:

$$\begin{aligned}\mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}[\log p_{\boldsymbol{\theta}}(\mathbf{x})] &= \\ &= \mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}\left[\mathbb{E}_{\boldsymbol{\zeta}\sim p_{\boldsymbol{\theta}}(\boldsymbol{\zeta}|\mathbf{x})}\left[\log \frac{p_{\boldsymbol{\theta}}(\mathbf{x}, \boldsymbol{\zeta})}{p_{\boldsymbol{\theta}}(\boldsymbol{\zeta}|\mathbf{x})}\right]\right] \,.\end{aligned} \tag{2}$$

By noticing that $p_{\boldsymbol{\theta}}(\boldsymbol{\zeta}, \mathbf{x}) = p_{\boldsymbol{\theta}}(\boldsymbol{\zeta}) p_{\boldsymbol{\theta}}(\mathbf{x}|\boldsymbol{\zeta})$ and rearranging Eq. 2, we have:

$$\begin{aligned}\mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}[\log p_{\boldsymbol{\theta}}(\mathbf{x})] = \mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}\Bigg[&\mathbb{E}_{\boldsymbol{\zeta}\sim p_{\boldsymbol{\theta}}(\boldsymbol{\zeta}|\mathbf{x})}[\log p_{\boldsymbol{\theta}}(\mathbf{x}|\boldsymbol{\zeta})] \\ &- \underbrace{\mathbb{E}_{\boldsymbol{\zeta}\sim p_{\boldsymbol{\theta}}(\boldsymbol{\zeta}|\mathbf{x})}\left[\log \frac{p_{\boldsymbol{\theta}}(\boldsymbol{\zeta}|\mathbf{x})}{p_{\boldsymbol{\theta}}(\boldsymbol{\zeta})}\right]}_{D_{KL}(p_{\boldsymbol{\theta}}(\boldsymbol{\zeta}|\mathbf{x})||p_{\boldsymbol{\theta}}(\boldsymbol{\zeta}))}\Bigg] \,,\end{aligned} \tag{3}$$



(a) Undirected generative model with latent variables

(b) Directed generative model (VAE)

(c) DVAE or QVAE

FIG. 1: Generative models with latent variables can be represented as graphical models that describe conditional relationships within variables. a) Undirected generative models are defined in terms of the a joint probability distribution, $p_{\boldsymbol{\theta}}(\mathbf{x},\boldsymbol{\zeta})$. Boltzmann machines belong to this group of generative models. b) In a directed generative model, the joint probability distribution $p_{\boldsymbol{\theta}}(\mathbf{x},\boldsymbol{\zeta})$, is decomposed as $p_{\boldsymbol{\theta}}(\mathbf{x},\boldsymbol{\zeta}) = p_{\boldsymbol{\theta}}(\mathbf{x}|\boldsymbol{\zeta})p_{\boldsymbol{\theta}}(\boldsymbol{\zeta})$: a prior distribution over the latent variables $p_{\boldsymbol{\theta}}(\boldsymbol{\zeta})$ and a decoder distribution $p_{\boldsymbol{\theta}}(\mathbf{x}|\boldsymbol{\zeta})$. The prior and the decoder are "hard-coded" or explicitly determined by the model; however, the posterior, $p_{\boldsymbol{\theta}}(\boldsymbol{\zeta}|\mathbf{x})$ (dotted red arrow) is intractable. In VAEs, an approximating posterior, $q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x})$, is proposed to replace the intractable true posterior. c) Structure of the generative and the inference (red dotted arrows) models of a DVAE and a QVAE. Here, $p_{\boldsymbol{\theta}}(\mathbf{z})$ represents the prior over discrete variables, $\mathbf{z}$, and is characterized by an RBM or a QBM in DVAEs or QVAEs, respectively. The continuous variables $\boldsymbol{\zeta}$ are introduced to allow for a smooth propagation of the gradients.

where $p_{\boldsymbol{\theta}}(\boldsymbol{\zeta}) = \sum_{\mathbf{x}} p_{\boldsymbol{\theta}}(\mathbf{x},\boldsymbol{\zeta})$ is the *prior* distribution. The term $D_{KL}(p||q) \equiv \mathbb{E}_p \log[p/q]$ represents the Kullback-Leibler (KL) divergence, which is a measure of "distance" between the two distributions p and q [67].

Maximizing the first term maximizes the probability of $p_{\boldsymbol{\theta}}(\mathbf{x}|\boldsymbol{\zeta})$ when $\boldsymbol{\zeta}$ is sampled from $p_{\boldsymbol{\theta}}(\boldsymbol{\zeta}|\mathbf{x})$ for a given input from the dataset. This is called reconstruction, because it implies that samples from $p_{\boldsymbol{\theta}}(\mathbf{x}|\boldsymbol{\zeta})$ have maximum similarity to the input $\mathbf{x}$. This is an "autoencoding" process and hence the first term in Eq. (3) is the autoencoding term. Conversely, maximizing the second term, corresponds to minimizing the expected KL divergence under p_{data}. For a given input $\mathbf{x}$, this amounts to minimizing the distance between the posterior $p_{\boldsymbol{\theta}}(\boldsymbol{\zeta}|\mathbf{x})$ and the prior $p_{\boldsymbol{\theta}}(\boldsymbol{\zeta})$. In the limiting case, this leads to $p_{\boldsymbol{\theta}}(\boldsymbol{\zeta}|\mathbf{x}) = p_{\boldsymbol{\theta}}(\boldsymbol{\zeta})$, which is only possible if $p_{\boldsymbol{\theta}}(\mathbf{x},\boldsymbol{\zeta}) = p_{\boldsymbol{\theta}}(\boldsymbol{\zeta})p_{\boldsymbol{\theta}}(\mathbf{x})$. Therefore, the *mutual information* [68] between $\mathbf{x}$ and $\boldsymbol{\zeta}$ is zero. In other words, while the autoencoding term strives to maximize the mutual information, the KL term tries to minimize it.

Eventually, the amount of information condensed in the latent space depends on the intricate balance between the two terms in Eq. 3, which in turn depends on the type of generative model chosen and also on the training method used. For example, directed models such as those depicted in Fig. 1(b) are characterized in terms of explicitly defining the prior $p_{\boldsymbol{\theta}}(\boldsymbol{\zeta})$ and the decoder $p_{\boldsymbol{\theta}}(\mathbf{x}|\boldsymbol{\zeta})$ distributions. If the decoder distribution has high representation power, it can easily decouple $\mathbf{x}$ and $\boldsymbol{\zeta}$ to avoid paying the KL penalty. This leads to poor reconstruction quality. On the other hand, if the decoder is less expressive (as is a neural net yielding the parameters of a factorial Bernoulli distribution) a high amount of information is stored in the latent space and the model autoencodes to a good degree.

III. VARIATIONAL AUTOENCODERS

A common problem of generative models with latent variables is the intractability of *inference*, *i.e.*, calculating the posterior distribution $p_{\boldsymbol{\theta}}(\boldsymbol{\zeta}|\mathbf{x}) = p_{\boldsymbol{\theta}}(\mathbf{x}|\boldsymbol{\zeta})p_{\boldsymbol{\theta}}(\boldsymbol{\zeta})/p_{\boldsymbol{\theta}}(\mathbf{x})$. This involves the evaluation of

$$p_{\boldsymbol{\theta}}(\mathbf{x}) = \int p_{\boldsymbol{\theta}}(\mathbf{x}|\boldsymbol{\zeta})p_{\boldsymbol{\theta}}(\boldsymbol{\zeta})d\boldsymbol{\zeta}\,. \tag{4}$$

The first crucial element of the VAE setup is variational inference; *i.e.*, introducing a tractable variational approximation $q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x})$ (Fig. 1(b)) to the true posterior $p_{\boldsymbol{\theta}}(\boldsymbol{\zeta}|\mathbf{x})$ [69], with variational parameters $\boldsymbol{\phi}$. Both decoder $p_{\boldsymbol{\theta}}(\mathbf{x}|\boldsymbol{\zeta})$ and encoder $q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x})$ are commonly implemented by neural networks, known as generative and recognition (inference) networks, respectively.

To define an objective function for optimizing parameters $\boldsymbol{\theta}$ and $\boldsymbol{\phi}$, we can replace $p_{\boldsymbol{\theta}}(\boldsymbol{\zeta}|\mathbf{x})$ with $q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x})$ in Eq. 3:

$$\begin{aligned}\mathcal{L}(\boldsymbol{\theta},\boldsymbol{\phi}) &\equiv \mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}[\mathcal{L}(\boldsymbol{\theta},\boldsymbol{\phi},\mathbf{x})] \equiv \\ &= \mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}[\mathbb{E}_{\boldsymbol{\zeta}\sim q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x})}[\log p_{\boldsymbol{\theta}}(\mathbf{x}|\boldsymbol{\zeta})] + \\ &- D_{KL}(q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x})||p_{\boldsymbol{\theta}}(\boldsymbol{\zeta}))]\,.\end{aligned} \tag{5}$$

Although $\mathcal{L}(\boldsymbol{\theta},\boldsymbol{\phi})$ is not equal to the log-likelihood, it provides a lower bound:

$$\mathcal{L}(\boldsymbol{\theta},\boldsymbol{\phi}) \leq \mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}[\log p_{\boldsymbol{\theta}}(\mathbf{x})]\,, \tag{6}$$

as we show below. Because of this important property, $\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi})$ is called the evidence (variational) lower bound (ELBO). To prove Eq. 6, we note from Eq. 5 that

$$\begin{aligned}\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}, \mathbf{x}) &= \mathbb{E}_{\boldsymbol{\zeta}\sim q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x})}\left[\log p_{\boldsymbol{\theta}}(\mathbf{x}|\boldsymbol{\zeta}) - \log \frac{q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x})}{p_{\boldsymbol{\theta}}(\boldsymbol{\zeta})}\right] = \\ &= \mathbb{E}_{\boldsymbol{\zeta}\sim q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x})}\left[\log \frac{p_{\boldsymbol{\theta}}(\mathbf{x}, \boldsymbol{\zeta})}{q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x})}\right] \qquad (7)\end{aligned}$$

where we have used $p_{\boldsymbol{\theta}}(\mathbf{x}, \boldsymbol{\zeta}) = p_{\boldsymbol{\theta}}(\boldsymbol{\zeta})p_{\boldsymbol{\theta}}(\mathbf{x}|\boldsymbol{\zeta})$. Eq. 7 is a compact way of expressing the ELBO, which will be used later. One may further use $p_{\boldsymbol{\theta}}(\mathbf{x}, \boldsymbol{\zeta}) = p_{\boldsymbol{\theta}}(\mathbf{x})p_{\boldsymbol{\theta}}(\boldsymbol{\zeta}|\mathbf{x})$ to obtain yet another way of writing the ELBO:

$$\begin{aligned}\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}, \mathbf{x}) &= \log p_{\boldsymbol{\theta}}(\mathbf{x}) - \mathbb{E}_{\boldsymbol{\zeta}\sim q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x})}\left[\log \frac{q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x})}{p_{\boldsymbol{\theta}}(\boldsymbol{\zeta}|\mathbf{x})}\right] \\ &= \log p_{\boldsymbol{\theta}}(\mathbf{x}) - D_{KL}(q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x})||p_{\boldsymbol{\theta}}(\boldsymbol{\zeta}|\mathbf{x}))]. \qquad (8)\end{aligned}$$

Since KL divergence is always non-negative, we obtain

$$\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}, \mathbf{x}) \leq \log p_{\boldsymbol{\theta}}(\mathbf{x}), \qquad (9)$$

which immediately gives Eq. 6.

It is evident from Eq. 8 that the difference between the ELBO and the true log-likelihood, *i.e.*, the tightness of the bound, depends on the distance between the approximate and true posteriors. Maximizing the ELBO, therefore, increases the log-likelihood and decreases the distance between the two posterior distributions at the same time. Success in minimizing the bound between the log-likelihood and ELBO depends on the flexibility and representational power of $q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x})$. However, increasing the representational power of $q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x})$ does not guarantee success in encoding the information in the latent space. In other words, the widespread problem [70–74] of "ignoring the latent code" in VAEs is not *completely* an artifact of choosing a family of approximating posterior distributions with limited representational power. As we discussed before, it is rather an intrinsic feature of generative models with latent variables due to the clash of the two terms in the objective function defined in Eq. 3.

A. The reparameterization trick

The objective function in Eq. 7 contains expectation values of functions of the latent variables $\boldsymbol{\zeta}$ under the posterior distribution $q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x})$. To train the model, we need to calculate the derivatives of these terms with respect to $\boldsymbol{\theta}$ and $\boldsymbol{\phi}$. However, evaluating the derivatives with respect to $\boldsymbol{\phi}$ is problematic because the expectations of Eq. 7 are estimated using samples that are generated according to a probability distribution that depends on $\boldsymbol{\phi}$. A naive solution to the problem of calculating $\partial_{\boldsymbol{\phi}}$ of the expected value of an arbitrary function $\mathbb{E}_{\boldsymbol{\zeta}\sim q_\phi}[f(\boldsymbol{\zeta})]$, is to use the identity $\partial_{\boldsymbol{\phi}} q_\phi = q_\phi \partial_{\boldsymbol{\phi}} \log q_\phi$, to write

$$\partial_{\boldsymbol{\phi}}\mathbb{E}_{\boldsymbol{\zeta}\sim q_\phi}\left[f(\boldsymbol{\zeta})\right] = \mathbb{E}_{\boldsymbol{\zeta}\sim q_\phi}\left[f(\boldsymbol{\zeta})\partial_{\boldsymbol{\phi}} \log q_\phi\right]. \qquad (10)$$

Here, for simplicity we assumed that f does not depend on $\boldsymbol{\phi}$. This approach is known as the REINFORCE. However, the expectation of Eq. 10 has high variance and requires intricate variance-reduction mechanisms to be of practical use [75].

A better approach is to write the random variable $\boldsymbol{\zeta}$ as a deterministic function of the distribution parameters $\boldsymbol{\phi}$ and of an additional auxiliary random variable $\boldsymbol{\rho}$. The latter is given by a probability distribution $p(\boldsymbol{\rho})$ that does not depend on $\boldsymbol{\phi}$. This reparameterization, $\boldsymbol{\zeta}(\boldsymbol{\phi}, \boldsymbol{\rho})$, can be used to write $\mathbb{E}_{\boldsymbol{\zeta}\sim q_\phi}[f(\boldsymbol{\zeta})] = \mathbb{E}_{\boldsymbol{\rho}\sim p(\boldsymbol{\rho})}[f(\boldsymbol{\zeta}(\boldsymbol{\phi}, \boldsymbol{\rho}))]$. Therefore, we can move the derivative inside the expectation with no difficulty:

$$\partial_{\boldsymbol{\phi}}\mathbb{E}_{\boldsymbol{\zeta}\sim q_\phi}\left[f(\boldsymbol{\zeta})\right] = \mathbb{E}_{\boldsymbol{\rho}\sim p(\boldsymbol{\rho})}\left[\partial_{\boldsymbol{\phi}} f(\boldsymbol{\zeta}(\boldsymbol{\phi}, \boldsymbol{\rho}))\right]. \qquad (11)$$

This is called the *reparameterization trick* [53] and is mostly responsible for the recent success and proliferation of VAEs. When applied to Eq. 7, we have:

$$\begin{aligned}\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}, \mathbf{x}) &= \mathbb{E}_{\boldsymbol{\zeta}\sim q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x})}\left[\log \frac{p_{\boldsymbol{\theta}}(\mathbf{x}, \boldsymbol{\zeta})}{q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x})}\right] \\ &= \mathbb{E}_{\boldsymbol{\rho}\sim p(\boldsymbol{\rho})}\left[\log \frac{p_{\boldsymbol{\theta}}(\mathbf{x}, \boldsymbol{\zeta}(\boldsymbol{\phi}, \boldsymbol{\rho}))}{q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}(\boldsymbol{\phi}, \boldsymbol{\rho})|\mathbf{x})}\right], \qquad (12)\end{aligned}$$

where we have suppressed the inclusion of $\mathbf{x}$ in the arguments of the reparameterized $\boldsymbol{\zeta}$ to keep the notation uncluttered.

It is now important to find a function $\boldsymbol{\zeta}(\boldsymbol{\phi}, \boldsymbol{\rho})$ such that $\boldsymbol{\rho}$ becomes $\boldsymbol{\phi}$-independent. Let us define a function $\mathbf{F}$

$$\boldsymbol{\rho} \equiv \mathbf{F}_{\boldsymbol{\phi}}(\boldsymbol{\zeta}). \qquad (13)$$

The probability distributions $p(\boldsymbol{\rho})$ and $q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x})$ should satisfy $p(\boldsymbol{\rho})d\boldsymbol{\rho} = q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x})d\boldsymbol{\zeta}$, therefore

$$p(\boldsymbol{\rho}) = \frac{q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x})}{d\boldsymbol{\rho}/d\boldsymbol{\zeta}} = \frac{q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x})}{d\mathbf{F}_{\boldsymbol{\phi}}(\boldsymbol{\zeta})/d\boldsymbol{\zeta}}. \qquad (14)$$

To have $p(\boldsymbol{\rho})$ independent of $\boldsymbol{\phi}$ we need

$$\mathbf{F}_{\boldsymbol{\phi}}(\boldsymbol{\zeta}) = \int_0^{\boldsymbol{\zeta}} q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}'|\mathbf{x})d\boldsymbol{\zeta}'. \qquad (15)$$

Now, by choosing $\mathbf{F}_{\boldsymbol{\phi}}$ to be the cumulative distribution function (CDF) of $q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x})$, $p(\boldsymbol{\rho})$ becomes a uniform distribution $\mathcal{U}(0, 1)$ for $\boldsymbol{\rho} \in [0, 1]$. We can thus write

$$\boldsymbol{\zeta}(\boldsymbol{\phi}, \boldsymbol{\rho}) = F_{\boldsymbol{\phi}}^{-1}(\boldsymbol{\rho}). \qquad (16)$$

To derive Eq. 16, we have implicitly assumed that the latent variables are continuous and that the posterior factorizes: $q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x}) = \prod_l q_{\boldsymbol{\phi}}(\zeta_l|\mathbf{x})$. It is possible to extend the reparameterization trick to include discrete latent variables (see next section) and more complicated approximate posteriors (see Appendix C).

IV. VAE WITH DISCRETE LATENT SPACE

Most of the VAEs studied so far have continuous latent spaces due to the difficulty of propagating derivatives through discrete variables. Nonetheless, discrete stochastic units are indispensable to representing distributions in supervised and unsupervised learning, attention models, language modeling and reinforcement learning [76]. Some noteworthy examples include application of discrete units in learning distinct semantic classes [62] and in semisupervised generation [77] to learn more meaningful hierarchical VAEs. In Ref. [78], when the latent space is composed of discrete variables, the representations learn to disentangle content and style information of images in an unsupervised fashion.

Due to the non-differentiability of discrete stochastic units, several methods that involve variational inference use the REINFORCE method, Eq. 10, from the reinforcement learning literature [75, 79, 80]. However, these methods yield noisy estimates of gradients that need to be mitigated using several variance reduction techniques such as finding appropriate control variates. Another approach involves using biased derivatives for the Bernoulli variables [81]. There are also two approaches that extend the reparameterization trick to discrete variables. Refs. [76, 82] concurrently came up with a relaxation of categorical discrete units into continuous variables by adding Gumbel noise to the logits inside a softmax function, with a temperature hyper-parameter. The softmax function transforms into a non-differentiable argmax function obtaining unbiased samples in the limit of zero temperature. However, in this limit the training stops since variables become truly discrete. Therefore, an annealing schedule is used for the temperature throughout the training to obtain less noisy, yet biased, estimates of gradients [76].

Here we follow the approach proposed in [55], which yields reparameterizable and unbiased estimates of gradients. As discussed in the previous section, the generative process in a VAE involves sampling a set of continuous variables $\boldsymbol{\zeta} \sim p_{\boldsymbol{\theta}}(\boldsymbol{\zeta})$. To implement a DVAE, we assume the prior distribution is now defined on a set of discrete variables $\mathbf{z} \sim p_{\boldsymbol{\theta}}(\mathbf{z})$, with $\mathbf{z} \in \{0,1\}^L$. Once again we use $\boldsymbol{\theta}$ to denote collective parameters of the generative side of the model. To propagate the gradients through the discrete variables, we keep the variables $\boldsymbol{\zeta}$ as an auxiliary set of continuous variables [55]. The full prior is chosen as follows (Fig. 1(c)) :

$$p_{\boldsymbol{\theta}}(\boldsymbol{\zeta}, \mathbf{z}) \equiv r(\boldsymbol{\zeta}|\mathbf{z}) p_{\boldsymbol{\theta}}(\mathbf{z}) \equiv \left(\prod_{l=1}^{L} r(\zeta_l|z_l) \right) p_{\boldsymbol{\theta}}(\mathbf{z}) \,. \tag{17}$$

The newly introduced term $r(\boldsymbol{\zeta}|\mathbf{z})$ acts as a smoothing probability distribution that enables the implementation of the reparameterization trick. The structure of the DVAE is completed by considering a particular form for the approximating posterior and marginal distributions (Fig. 1(c)):

$$\begin{aligned} q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}, \mathbf{z}|\mathbf{x}) &\equiv r(\boldsymbol{\zeta}|\mathbf{z}) q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}) \\ p_{\boldsymbol{\theta}}(\mathbf{x}|\boldsymbol{\zeta}, \mathbf{z}) &\equiv p_{\boldsymbol{\theta}}(\mathbf{x}|\boldsymbol{\zeta}) \,, \end{aligned} \tag{18}$$

where for now we assume $q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}) = \prod_l q_{\boldsymbol{\phi}}(z_l|\mathbf{x})$ is a product of Bernoulli probabilities for the discrete variable z_l (see again Appendix C for the case where hierarchies are present in the posterior). With the above choice, the ELBO bound can be written as

$$\begin{aligned} \mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}, \mathbf{x}) &= \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x})}[\log p_{\boldsymbol{\theta}}(\mathbf{x}|\boldsymbol{\zeta})] + \\ &- D_{KL}(q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})||p_{\boldsymbol{\theta}}(\mathbf{z})) \,, \end{aligned} \tag{19}$$

where $q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x})$ is the approximate posterior marginalized over the discrete variables. In the equation above we have used the fact that the KL term does not explicitly depend on $\boldsymbol{\zeta}$ while the autoencoding term does not explicitly depend on $\mathbf{z}$.

A. The reparameterization trick for DVAE

We can apply the inverse CDF reparameterization trick, Eq. 16, to the autoencoding term in Eq. 19 if we choose the function $r(\boldsymbol{\zeta}|\mathbf{z})$ such that the CDF of the approximating posterior marginalized over the discrete variables

$$\mathbf{F}(\boldsymbol{\zeta}) \equiv \int_0^{\boldsymbol{\zeta}} q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}'|\mathbf{x}) d\boldsymbol{\zeta}' \tag{20}$$

can be inverted:

$$\mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x})}[\log p_{\boldsymbol{\theta}}(\mathbf{x}|\boldsymbol{\zeta})] = \mathbb{E}_{\boldsymbol{\rho} \sim \mathcal{U}}[\log p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{F}^{-1}(\boldsymbol{\rho})] \,. \tag{21}$$

An appropriate choice for $r(\zeta_l|z_l)$ is, for example, the spike-and-exponential transformation:

$$\begin{aligned} r(\zeta_l|z_l=0) &= \delta(\zeta_l) \\ r(\zeta_l|z_l=1) &= \begin{cases} \beta \frac{e^{\beta\zeta_l}}{e^{\beta}-1}, & \text{if} \quad 0 < \zeta_l \leq 1 \\ 0, & \text{otherwise} \,. \end{cases} \end{aligned} \tag{22}$$

For this distribution we can write:

$$\mathrm{F}_l(\zeta_l) = \int_0^{\zeta_l} q_{\boldsymbol{\phi}}(\zeta_l'|\mathbf{x}) d\zeta_l' = \int_0^{\zeta_l} \sum_{z_l=0,1} q_{\boldsymbol{\phi}}(z_l|\mathbf{x}) r(\zeta_l'|z_l) d\zeta_l' \,. \tag{23}$$

Using Eq. 22 with Bernoulli distribution $q_{\boldsymbol{\phi}}(z_l{=}1|\mathbf{x}) = q_l$ and $q_{\boldsymbol{\phi}}(z_l{=}0|\mathbf{x}) = 1{-}q_l$, we find

$$\rho_l = q_l \frac{e^{\beta\zeta_l} - 1}{e^{\beta} - 1} + (1 - q_l) \,, \tag{24}$$

which can be easily inverted to obtain ζ_l

$$\zeta_l(\rho_l, q_l) = \frac{1}{\beta} \log \left[\left(\frac{\max(\rho_l + q_l - 1, 0)}{q_l} \right) (e^{\beta} - 1) + 1 \right] . \tag{25}$$

The virtue of the spike-and-exponential smoothing distribution is that z_l can be deterministically obtained from ζ_l and thus ρ_l:

$$z_l(\rho_l, q_l) = \text{sign}(\zeta_l(\rho_l, q_l)) = \Theta(\rho_l + q_l - 1)\,, \tag{26}$$

which follows from Eqs 22 and 25. This property is crucial to apply the reparameterization trick to the KL term, as shown below, and evaluating its derivatives as shown in Appendix D.

For later convenience, we note that the KL term can be written as the difference between an *entropy* term, $H(q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}))$, and a *cross-entropy* term, $H(q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}), p_{\boldsymbol{\theta}}(\mathbf{z}))$:

$$D_{KL}(q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})||p_{\boldsymbol{\theta}}(\mathbf{z})) = \underbrace{\mathbb{E}_{q_{\boldsymbol{\phi}}}[\log q_{\boldsymbol{\phi}}]}_{-H(q_{\boldsymbol{\phi}})} - \underbrace{\mathbb{E}_{q_{\boldsymbol{\phi}}}[\log p_{\boldsymbol{\theta}}]}_{-H(q_{\boldsymbol{\phi}}, p_{\boldsymbol{\theta}})}\,.$$

Herein, for simplicity, we use q_ϕ and p_θ in place of $q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})$ and $p_{\boldsymbol{\theta}}(\mathbf{z})$, respectively, in unambiguous cases. Using Eq. 26, the reparameterization trick can be applied to the entropy term:

$$H(q_{\boldsymbol{\phi}}) \equiv -\mathbb{E}_{\mathbf{z}\sim q_{\boldsymbol{\phi}}}[\log q_{\boldsymbol{\phi}}] = -\mathbb{E}_{\boldsymbol{\rho}\sim\mathcal{U}}[\log q_{\boldsymbol{\phi}}(\mathbf{z}(\boldsymbol{\rho}, \boldsymbol{\phi})|\mathbf{x})], \tag{27}$$

where we have explicitly shown the dependence of $\mathbf{z}$ on $\boldsymbol{\rho}$ and $\boldsymbol{\phi}$. Note that in the simple case of a factorial Bernoulli distribution, we do not need to use the reparameterization trick and can use the analytic form of the entropy; *i.e.*, $H(q_{\boldsymbol{\phi}}) = -\sum_{l=1}^{L}(q_l \log q_l + (1-q_l)\log(1-q_l))$ (see Appendix B for more details). Similarly, applying the reparameterization trick to the cross-entropy leads to:

$$-H(q_{\boldsymbol{\phi}}, p_{\boldsymbol{\theta}}) \equiv \mathbb{E}_{\mathbf{z}\sim q_{\boldsymbol{\phi}}}[\log p_{\boldsymbol{\theta}}] = \mathbb{E}_{\boldsymbol{\rho}\sim\mathcal{U}}[\log p_{\boldsymbol{\theta}}(\mathbf{z}(\boldsymbol{\rho}, \boldsymbol{\phi}))]\,. \tag{28}$$

It is a common practice to use hierarchical distributions to achieve more powerful approximating posteriors. Briefly, the latent variables are compartmentalized into several groups, and the probability density function of each group depends on the values of the latent variables in the preceding groups; *i.e.*, $q_{\boldsymbol{\phi}}(z_l|\zeta_{m<l}, \mathbf{x})$. This creates a more powerful approximating posterior able to represent more complex correlations between latent variables, as compared to a simple factorial distribution. See Appendix C for more details.

B. DVAE with Boltzmann machines

Boltzmann machines are probabilistic models able to represent complex multi-modal probability distributions [83], and are thus attractive candidates for the latent space of a VAE. This approach is also appealing with regards to the machine-learning application of quantum computers. The probability distribution realized by an RBM is

$$\begin{aligned} p_{\boldsymbol{\theta}}(\mathbf{z}) &\equiv e^{-E_{\boldsymbol{\theta}}(\mathbf{z})}/Z_{\boldsymbol{\theta}}\,, \quad Z_{\boldsymbol{\theta}} \equiv \sum_{\mathbf{z}} e^{-E_{\boldsymbol{\theta}}(\mathbf{z})}\,, \\ E_{\boldsymbol{\theta}}(\mathbf{z}) &= \sum_l z_l h_l + \sum_{l<m} W_{lm} z_l z_m, \quad \mathbf{h}, \mathbf{W} \in \{\boldsymbol{\theta}\}. \end{aligned} \tag{29}$$

The negative cross entropy term $-H(q_{\boldsymbol{\phi}}, p_{\boldsymbol{\theta}}) = \mathbb{E}_{\mathbf{z}\sim q_{\boldsymbol{\phi}}}[\log p_{\boldsymbol{\theta}}]$ is the log-likelihood of $\mathbf{z}$ sampled from the approximating posterior $\mathbf{z} \sim q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})$ under the model $p_{\boldsymbol{\theta}}$. After reparameterization, we have

$$\begin{aligned} H(q_{\boldsymbol{\phi}}, p_{\boldsymbol{\theta}}) &= -\mathbb{E}_{\boldsymbol{\rho}\sim\mathcal{U}}[\log p_{\boldsymbol{\theta}}(\mathbf{z}(\boldsymbol{\rho}, \boldsymbol{\phi}))] \\ &= \mathbb{E}_{\boldsymbol{\rho}\sim\mathcal{U}}[E_{\boldsymbol{\theta}}(\mathbf{z}(\boldsymbol{\rho}, \boldsymbol{\phi}))] + \log Z_{\boldsymbol{\theta}}. \end{aligned} \tag{30}$$

Gradients can thus be computed as usual as the difference between a positive and negative phase, in which the latter is computed via Boltzmann sampling from the BM:

$$\partial H(q_{\boldsymbol{\phi}}, p_{\boldsymbol{\theta}}) = \mathbb{E}_{\boldsymbol{\rho}\sim\mathcal{U}}[\partial E_{\boldsymbol{\theta}}(\mathbf{z}(\boldsymbol{\rho}, \boldsymbol{\phi}))] - \mathbb{E}_{\mathbf{z}\sim p_{\boldsymbol{\theta}}}[\partial E_{\boldsymbol{\theta}}(\mathbf{z})]\,. \tag{31}$$

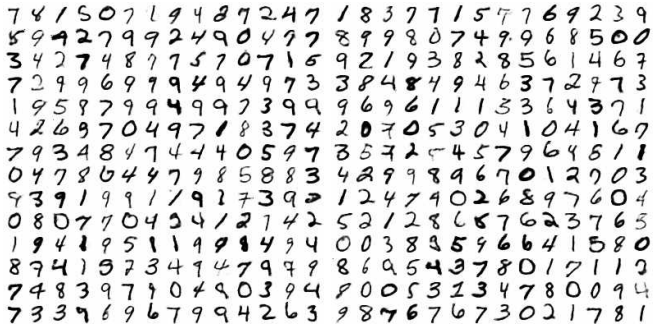
Notice that the positive phase (the first term above) involves the expectation over the approximating posterior, but it is explicitly written in terms of the discrete variables $\mathbf{z}(\boldsymbol{\rho}, \boldsymbol{\phi})$. We thus need to calculate the derivatives through these variables. We discuss the computation of the positive phase in the most general case in Appendix D.

C. Experimental results with DVAE

In this section, we show that the DVAE model introduced in Sec. IV achieves state-of-the-art performance, for variational inference models with only latent variables, on the MNIST dataset [48]. We perform experiments with restricted Boltzmann machines, in which the hidden and visible units are placed at the two sides of a bipartite graph. Notice that in a DVAE setup, all the units of the (classical) RBM are latent variables (there is technically no distinction between visible and hidden units as for standalone RBMs). We still use an RBM to exploit its bipartite structure enabling efficient Gibbs block-sampling. This allows us to train DVAEs with RBMs with up to 256 units per layer.

Figure 2 shows generated and reconstructed MNIST digits for a DVAE with RBMs with 32 and 256 units per layer. In Table I, we report the best results for the ELBO and log-likelihood (LL) we obtained with RBMs of $32, 64, 128$, and 256 units per layer. For 256 units, we obtained an LL of -83.5 ± 0.2, with the reported error being a conservative estimate of our statistical uncertainty. In all cases, the negative phase of the RBMs was estimated using persistent contrastive divergence (PCD), with 1000 chains and 200 block-Gibbs updates per gradient evaluation. We have chosen an approximating posterior with 8 levels of hierarchies (the number of units that each level of hierarchy represents is the total number of latent units divided by 8); each Bernoulli probability $q_{\boldsymbol{\phi}}(z_l|\zeta_{m<l}, \mathbf{x})$ is

(a) Generated MNIST: DVAE, $\text{RBM}_{32\times32}$ (b) Reconstructed MNIST: DVAE, $\text{RBM}_{32\times32}$



(c) Generated MNIST: DVAE, $\text{RBM}_{256\times256}$ (d) Reconstructed MNIST: DVAE, $\text{RBM}_{256\times256}$

FIG. 2: Generated and reconstructed MNIST with DVAE.

a sigmoidal output of a feed-forward neural network with two hidden rectified linear unit (ReLU) layers containing 2000 deterministic units.

The model is prone to overfitting when representing the decoder distribution $p_{\boldsymbol{\theta}}(\mathbf{x}|\boldsymbol{\zeta})$ with deep networks. We considered $p_{\boldsymbol{\theta}}(\mathbf{x}|\boldsymbol{\zeta})$ to be sigmoidal outputs of a ReLu network with one layer and the number of deterministic units that vary between 250 and 2000. Typically, a larger RBM required a smaller number of hidden units in the decoder network to avoid overfitting. Our implementation included annealing schedules for both the learning rate (exponential decay) and the β parameter (linear increase) in Eq. 22. Batch normalization [84] was used to expedite the training process. The value of β was annealed throughout the training from 1.0 to 10 during 2000 epochs with a batch size of 200. We used the ADAM stochastic optimization method with a learning rate of 10^{-3} and the default extra parameters [85]. To calculate the LL in Table I, we used importance weighting to compute a multi-sample ELBO, as delineated in [54], with 30000 samples in the latent space for each input image in the test set. It can be shown that the value of the multi-sample ELBO asymptotically reaches the true LL when the number of samples approaches infinity [54]. The $\log Z$ was computed using population annealing [86, 87] (see also Appendix E for the quantum partition function). In all our experiments we have verified that the statistical error on the evaluation of $\log Z$ is negligible.

MNIST (static binarization)			
		ELBO	**LL**
DVAE	$\text{RBM}_{32\times32}$	-99.3 ± 0.2	-90.8 ± 0.2
	$\text{RBM}_{64\times64}$	-92.4	-85.5
	$\text{RBM}_{128\times128}$	-90.4	-84.7
	$\text{RBM}_{256\times256}$	$\mathbf{-89.2}$	$\mathbf{-83.5}$
VIMCO [88]			-91.9
NVIL [75]			-93.5
CONCRETE [82]			-85.7
GS [76]		-101.5	
RWS [89]			-88.9
REBAR [90]		-98.8	

TABLE I: Comparison of variational generative models with stochastic discrete variables on the validation set of the MNIST dataset. The best results are denoted by boldface font. GS stands for Gumbel-Softmax. The confidence level for the DVAE results is smaller than ± 0.2 in all cases.

In Table I, we also report the results of some other algorithms that use discrete variables in variational inference. NVIL [75] and its importance weighted analog, VIMCO [88], use the REINFORCE trick, Eq. 10 along with carefully designed control variates to reduce the variance of the estimation. CONCRETE [82] and Gumbel-Softmax [76] are two concurrently developed methods that are based on applying the reparameterization trick to discrete latent variables. RWS [89] is a multi-sampled and improved version of the wake-sleep algorithm [8], which can be considered as a variational approximation (since an encoder or "inference network" is present) with different loss functions in the wake and sleep phases of training. REBAR [90] is the application of the CONCRETE method to create control variates for the REINFORCE approach. All algorithms reported in Table I, excluding DVAE, implement a latent space with independent discrete units distributed according to a set of independent Bernoulli distributions. The result reported for CONCRETE, for example, includes 200 independent latent units. The presence of a well-trained RBM in the latent space of DVAE is critical to achieve the results quoted in Table I. In particular, our implementation of DVAE is able to match the result obtained with the CONCRETE method by using only 64+64 latent units rather than 200. A direct demonstration of the necessity to have a well-trained RBM to achieve state-of-the-art performance with DVAE is also given in Table 2 of Ref. [91].

V. QUANTUM VARIATIONAL AUTOENCODERS

We now introduce the QVAE by implementing the prior distribution in the latent space of a VAE as a QBM.

(a) Generated MNIST: QVAE, $\text{QBM}_{16\times16}$, $\Gamma = 0$, (b) Generated MNIST: QVAE, $\text{QBM}_{64\times64}$, $\Gamma = 0$,

(c) Generated MNIST: QVAE, $\text{QBM}_{16\times16}$, $\Gamma = 1$, (d) Generated MNIST: QVAE, $\text{QBM}_{64\times64}$, $\Gamma = 1$,

(e) Generated MNIST: QVAE, $\text{QBM}_{16\times16}$, $\Gamma = 2$, (f) Generated MNIST: QVAE, $\text{QBM}_{64\times64}$, $\Gamma = 2$,

FIG. 3: Comparison of generated MNIST digits with different values of the transverse field Γ at the same stage of training (90 epochs).

Similar to a classical BM, a QBM is an energy model defined as follows [51]:

$$\begin{aligned} p_{\boldsymbol{\theta}}(\mathbf{z}) &\equiv \text{Tr}[\Lambda_{\mathbf{z}} e^{-\mathcal{H}_{\boldsymbol{\theta}}}]/Z_{\boldsymbol{\theta}}\,, \quad Z_{\boldsymbol{\theta}} \equiv \text{Tr}[e^{-\mathcal{H}_{\boldsymbol{\theta}}}]\,, \\ \mathcal{H}_{\boldsymbol{\theta}} &= \sum_l \sigma_l^x \Gamma_l + \sum_l \sigma_l^z h_l + \\ &\quad + \sum_{l<m} W_{lm} \sigma_l^z \sigma_m^z, \quad \boldsymbol{\Gamma}, \mathbf{h}, \mathbf{W} \in \{\boldsymbol{\theta}\}\,, \end{aligned} \tag{32}$$

where $\Lambda_{\mathbf{z}} \equiv |\mathbf{z}\rangle\langle\mathbf{z}|$ is the projector on the classical state $\mathbf{z}$ and $\sigma_l^{x,z}$ are Pauli operators. States $\mathbf{z}$ are distributed according to $p_{\boldsymbol{\theta}}(\mathbf{z})$; *e.g.*, a quantum Boltzmann distribution

MNIST (static binarization)			**ELBO**	**Q-ELBO**
QVAE:	$\Gamma = 0$	$\text{RBM}_{16\times16}$	-109.3 ± 0.2	-109.3 ± 0.2
	$\Gamma = 1$	$\text{QBM}_{16\times16}$	-110.5	-120.6
	$\Gamma = 2$		-115.3	-135.8
QVAE:	$\Gamma = 0$	$\text{RBM}_{32\times32}$	-101.8	-101.8
	$\Gamma = 1$	$\text{QBM}_{32\times32}$	-103.6	-117.9
	$\Gamma = 2$		-112.1	-139.7
QVAE:	$\Gamma = 0$	$\text{RBM}_{64\times64}$	-105.7	-105.7
	$\Gamma = 1$	$\text{QBM}_{64\times64}$	-108.7	-133.9
	$\Gamma = 2$		-120.0	-165.2

TABLE II: Evaluation on validation set: $\text{RBM}_{16\times16}$ at 800 epochs, $\text{RBM}_{32\times32}$ at 250 epochs, $\text{RBM}_{64\times64}$ at 50 epochs. The confidence level for the numerical results is smaller than ±0.2 in all cases.

for the quantum system given by $\mathcal{H}_{\boldsymbol{\theta}}$. Similar to the classical case, the ELBO includes the following cross-entropy term:

$$H(q_{\boldsymbol{\phi}}, p_{\boldsymbol{\theta}}) = -\mathbb{E}_{\mathbf{z}\sim q_{\boldsymbol{\phi}}}[\log(\text{Tr}[\Lambda_{\mathbf{z}} e^{-\mathcal{H}_{\boldsymbol{\theta}}}])] + \log Z_{\boldsymbol{\theta}}\,. \tag{33}$$

Unfortunately, the gradients of the first term in the equation above are intractable. A QBM can still be trained using a lower-bound to the cross-entropy that can be obtained via the Golden-Thompson inequality [51]:

$$\text{Tr}[e^A e^B] \geq \text{Tr}[e^{A+B}]\,, \tag{34}$$

which holds for any two Hermitian matrices. The equality is satisfied if and only if the two matrices commute. Using this inequality we can write for the cross-entropy:

$$\begin{aligned} H(q_{\boldsymbol{\phi}}, p_{\boldsymbol{\theta}}) &\geq \overbrace{-\mathbb{E}_{\mathbf{z}\sim q_{\boldsymbol{\phi}}}[\log(\text{Tr}[e^{-\mathcal{H}_{\boldsymbol{\theta}} + \ln \Lambda_{\mathbf{z}}}])] + \log Z_{\boldsymbol{\theta}}}^{\tilde{H}(q_{\boldsymbol{\phi}}, p_{\boldsymbol{\theta}})} \\ &= \mathbb{E}_{\boldsymbol{\rho}\sim\mathcal{U}}[\mathcal{H}_{\boldsymbol{\theta}}(\mathbf{z}(\boldsymbol{\rho}, \boldsymbol{\phi}))] + \log Z_{\boldsymbol{\theta}}, \end{aligned} \tag{35}$$

where in the second line we have used the reparameterization trick and the fact that the contribution to the trace of all states different than $\mathbf{z}$ is infinitely suppressed. In the equation above we have defined $\mathcal{H}_{\boldsymbol{\theta}}(\mathbf{z}) \equiv \langle\mathbf{z}|\, \mathcal{H}_{\boldsymbol{\theta}}\, |\mathbf{z}\rangle$. Using the lower-bound $\tilde{H}(q_{\boldsymbol{\phi}}, p_{\boldsymbol{\theta}})$, we obtain a tractable quantum bound (Q-ELBO) to the true ELBO and the QVAE can be trained by estimating the gradients via sampling from the QBM [51]:

$$\partial \tilde{H}(q_{\boldsymbol{\phi}}, p_{\boldsymbol{\theta}}) = \mathbb{E}_{\boldsymbol{\rho}\sim\mathcal{U}}[\partial \mathcal{H}_{\boldsymbol{\theta}}(\mathbf{z}(\boldsymbol{\rho}, \boldsymbol{\phi}))] - \mathbb{E}_{\mathbf{z}\sim p_{\theta}}[\partial \mathcal{H}_{\boldsymbol{\theta}}(\mathbf{z})]\,. \tag{36}$$

The use of the Q-ELBO and its gradients precludes the training of the transverse fields $\boldsymbol{\Gamma}$ [51], which is treated as a constant (hyper-parameter) throughout the training.

A. Experimental results with QVAE

In this section, we show that QVAEs can be effectively trained via the looser quantum bound (Q-ELBO). While

it is computationally unfeasible to train a QVAE that has a large QBM in the latent space with QMC, sampling from large QBMs is possible with the use of quantum annealing devices. Given the results of the previous and present sections, we thus expect the possibility of using quantum annealing devices to sample from large QBMs in the latent space of QVAEs to achieve competitive results on datasets such as MNIST.

To perform experiments with QVAEs, we have considered exactly the same models used in the case of DVAEs, exchanging the RBMs with QBMs. As explicated by Eqs. 35 and 36, we train the VAE by maximizing the lower bound Q-ELBO to the true ELBO. To compute the negative phase in Eq. 36, we have used population annealing (PA) for continuous-time quantum Monte Carlo (CT-QMC) (see Appendix E for more details). We have considered a population of 1000 samples and 5 sweeps per gradient evaluation. Despite being one of the most effective sampling methods we considered, population annealing CT-QMC is still numerically expensive and prevented us from fully training QVAEs with large QBMs. We thus considered (restricted) QBMs with 16, 32, and 64 units per layer and estimated the ELBO obtained with different values of the transverse field Γ. Table II shows the ELBO and Q-ELBO for different sizes of the QBMs and values of Γ. To estimate Q-ELBO, we use the classical energy function in the positive phase (with no transverse field) and a quantum partition function (see Appendix E). The ELBO is calculated using the true quantum probability of a state in the positive phase. The difference between ELBO and Q-ELBO is due to Eq. 34. We emphasize that the results of Table II correspond to ELBOs obtained at different stages during training (800, 250, and 50 epochs for the cases with 16, 32, and 64 units per layer, respectively). These numbers simply correspond to the largest number of epochs we were able to train each model during the preparation of this work. Also, note that we have chosen not report LL in this table, since this requires using importance sampling which is computationally expensive; this requires calculating the quantum probabilities for a large number of latent samples per input image.

Our results show, as expected, that Q-ELBO becomes looser as the transverse field is increased. Still, we observe that the corresponding ELBO we obtained during training is much tighter and closer to the classical case. This explain why we are able to effectively train QVAE with values of the transverse field as large as 2. We consider this value of the transverse field to be relatively large, since the typical scale of the trained couplings in the classical part of the Hamiltonian is of order 1. Figure 3 shows MNIST images generated with QVAE and CT-QMC sampling. We see that the quality of the generated samples is satisfactory for the considered values of the transverse field (up to $\Gamma = 2$).

It is important to stress that the deterioration of performance we observe in Table II is mostly due to that fact that the Q-ELBO used for training becomes looser as the transverse field increases. This is not necessarily an intrinsic limitation of QVAE. Indeed, in Ref. [51] it was shown that small quantum Boltzmann machines perform better than their classical counterparts if training is performed via direct maximization of the LL.

VI. CONCLUSIONS

We proposed a variational inference model, QVAE, that uses QBMs to implement the generative process in its latent space. We showed that this infrastructure can be powerful and at higher dimensions of latent space can give state-of-the-art results (for variational inference-based models with stochastic discrete units) on MNIST dataset. We used CT-QMC to sample from the QBMs in the latent space and were limited to smaller dimensions (up to a 64×64 dimensional QBM) due to computational cost. Introduction of QBMs in the latent space of our model introduces an additional quantum bound on the ELBO objective function of VAEs. However, we demonstrated empirically that QVAEs have generally similar performance to their classical limit where the transverse field is absent. On important open question for future work is whether it is possible to improve the performance of QVAE by using bounds to the LL that tighter than the Q-ELBO used in this work.

During training, both RBM and QBM develop well-defined modes that make sampling via Markov Chain Monte Carlo methods very inefficient. Quantum annealers could provide a computational advantage by exploiting quantum tunnelling to accelerate mixing between different modes. A computational advantage of this type was observed, with respect to quantum Monte Carlo methods, in Ref. [92]. The successful use of quantum annealers will likely require tailored implementations that mitigate physical limitations of actual devices such as control errors, limited coupling range and connectivity. This is a promising line of research which we are exploring for upcoming works.

This work is an attempt to add quantum algorithms to powerful existing classical frameworks to obtain new competitive generative models. It can be considered as a bedrock on which the next generation of quantum annealers can be efficiently used in order to solve realistic problems in machine learning.

Acknowledgements

The authors would like to thank Hossein Sadeghi, Arash Vahdat, and Jason T. Rolfe for useful discussions during the preparation of this work.

[1] F. Rosenblatt, Psychological review **65**, 386 (1958).
[2] D. E. Rumelhart, G. E. Hinton, R. J. Williams, et al., Cognitive modeling **5**, 1 (1988).
[3] G. E. Hinton, S. Osindero, and Y.-W. Teh, Neural computation **18**, 1527 (2006).
[4] Y. Bengio, P. Lamblin, D. Popovici, and H. Larochelle, in *Advances in neural information processing systems* (2007), pp. 153–160.
[5] Y. LeCun, Y. Bengio, and G. Hinton, Nature **521**, 436 (2015).
[6] I. Goodfellow, Y. Bengio, and A. Courville, http://www.deeplearningbook.org (2016).
[7] P. Vincent, H. Larochelle, Y. Bengio, and P.-A. Manzagol, in *Proceedings of the 25th international conference on Machine learning* (ACM, 2008), pp. 1096–1103.
[8] G. E. Hinton, P. Dayan, B. J. Frey, and R. M. Neal, Science **268**, 1158 (1995).
[9] T. Tieleman, in *Proceedings of the 25th international conference on Machine learning* (ACM, 2008), pp. 1064–1071.
[10] A. W. Harrow, A. Hassidim, and S. Lloyd, Physical review letters **103**, 150502 (2009).
[11] N. Wiebe, D. Braun, and S. Lloyd, Physical review letters **109**, 050505 (2012).
[12] A. M. Childs, R. Kothari, and R. D. Somma, arXiv preprint arXiv:1511.02306 (2015).
[13] S. Lloyd, M. Mohseni, and P. Rebentrost, Nature Physics **10**, 631 (2014).
[14] F. Dolde, V. Bergholm, Y. Wang, I. Jakobi, S. Pezzagna, J. Meijer, P. Neumann, T. Schulte-Herbrüggen, J. Biamonte, and J. Wrachtrup, arXiv preprint arXiv:1309.4430 (2013).
[15] E. Zahedinejad, J. Ghosh, and B. C. Sanders, Physical Review Applied **6**, 054005 (2016).
[16] U. Las Heras, U. Alvarez-Rodriguez, E. Solano, and M. Sanz, Physical review letters **116**, 230504 (2016).
[17] M. Schuld, I. Sinayskiy, and F. Petruccione, Contemporary Physics **56**, 172 (2015).
[18] P. Wittek, *Quantum machine learning: what quantum computing means to data mining* (Academic Press, 2014).
[19] J. Adcock, E. Allen, M. Day, S. Frick, J. Hinchliff, M. Johnson, S. Morley-Short, S. Pallister, A. Price, and S. Stanisic, arXiv preprint arXiv:1512.02900 (2015).
[20] S. Arunachalam and R. de Wolf, arXiv preprint arXiv:1701.06806 (2017).
[21] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, Nature **549**, 195 (2017), 1611.09347, URL https://arxiv.org/abs/1611.09347.
[22] M. A. Nielsen and I. Chuang, *Quantum computation and quantum information* (2002).
[23] D. A. Lidar and T. A. Brun, *Quantum error correction* (Cambridge University Press, 2013).
[24] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, Physical Review A **86**, 032324 (2012).
[25] C. Neill, P. Roushan, K. Kechedzhi, S. Boixo, S. Isakov, V. Smelyanskiy, R. Barends, B. Burkett, Y. Chen, Z. Chen, et al., arXiv preprint arXiv:1709.06678 (2017).
[26] A. Kandala, A. Mezzacapo, K. Temme, M. Takita, M. Brink, J. M. Chow, and J. M. Gambetta, Nature **549**, 242 (2017).
[27] E. Farhi, J. Goldstone, S. Gutmann, J. Lapan, A. Lundgren, and D. Preda, Science **292**, 472 (2001).
[28] M. W. Johnson, M. H. Amin, S. Gildert, T. Lanting, F. Hamze, N. Dickson, R. Harris, A. J. Berkley, J. Johansson, P. Bunyk, et al., Nature **473**, 194 (2011).
[29] E. Farhi, J. Goldstone, and S. Gutmann, arXiv preprint arXiv:1411.4028 (2014).
[30] A. Peruzzo, J. McClean, P. Shadbolt, M.-H. Yung, X.-Q. Zhou, P. J. Love, A. Aspuru-Guzik, and J. L. O'brien, Nature communications **5** (2014).
[31] J. Otterbach, R. Manenti, N. Alidoust, A. Bestwick, M. Block, B. Bloom, S. Caldwell, N. Didier, E. S. Fried, S. Hong, et al., arXiv preprint arXiv:1712.05771 (2017).
[32] T. Kadowaki and H. Nishimori, Physical Review E **58**, 5355 (1998).
[33] G. E. Santoro, R. Martoňák, E. Tosatti, and R. Car, Science **295**, 2427 (2002).
[34] J. Brooke, T. Rosenbaum, and G. Aeppli, Nature **413**, 610 (2001).
[35] M. H. Amin, Physical Review A **92**, 052323 (2015).
[36] L. C. Venuti, T. Albash, D. A. Lidar, and P. Zanardi, Physical Review A **93**, 032118 (2016).
[37] T. Albash, V. Martin-Mayor, and I. Hen, arXiv preprint arXiv:1703.03871 (2017).
[38] W. Vinci and D. A. Lidar, arXiv preprint arXiv:1710.07871 (2017).
[39] H. Neven, V. S. Denchev, G. Rose, and W. G. Macready, arXiv preprint arXiv:0811.0416 (2008).
[40] V. S. Denchev, N. Ding, S. Vishwanathan, and H. Neven, arXiv preprint arXiv:1205.1148 (2012).
[41] K. L. Pudenz and D. A. Lidar, Quantum information processing **12**, 2027 (2013).
[42] A. Mott, J. Job, J.-R. Vlimant, D. Lidar, and M. Spiropulu, Nature **550**, 375 (2017).
[43] M. Denil and N. De Freitas, in *NIPS 2011 Deep Learning and Unsupervised Feature Learning Workshop* (2011).
[44] J. Raymond, S. Yarkoni, and E. Andriyash, arXiv preprint arXiv:1606.00919 (2016).
[45] D. Korenkevych, Y. Xue, Z. Bian, F. Chudak, W. G. Macready, J. Rolfe, and E. Andriyash, arXiv preprint arXiv:1611.04528 (2016).
[46] A. Perdomo-Ortiz, M. Benedetti, J. Realpe-Gómez, and R. Biswas, arXiv preprint arXiv:1708.09757 (2017).
[47] S. H. Adachi and M. P. Henderson, arXiv preprint arXiv:1510.06356 (2015).
[48] Y. LeCun, http://yann. lecun. com/exdb/mnist/ (1998).
[49] M. Benedetti, J. Realpe-Gómez, R. Biswas, and A. Perdomo-Ortiz, Physical Review A **94**, 022308 (2016).
[50] M. Benedetti, J. Realpe-Gómez, R. Biswas, and A. Perdomo-Ortiz, arXiv preprint arXiv:1609.02542 (2016).
[51] M. H. Amin, E. Andriyash, J. Rolfe, B. Kulchytskyy, and R. Melko (2016), 1601.02036, URL https://arxiv.org/abs/1601.02036.
[52] M. Benedetti, J. Realpe-Gómez, and A. Perdomo-Ortiz, Quantum Science and Technology **3**, 034007 (2018).
[53] D. P. Kingma and M. Welling, arXiv preprint arXiv:1312.6114 (2013).
[54] Y. Burda, R. Grosse, and R. Salakhutdinov, arXiv preprint arXiv:1509.00519 (2015).
[55] J. T. Rolfe, arXiv preprint arXiv:1609.02200 (2016).
[56] J. Romero, J. P. Olson, and A. Aspuru-Guzik, Quantum

Science and Technology **2**, 045001 (2017).
[57] K. H. Wan, O. Dahlsten, H. Kristjánsson, R. Gardner, and M. Kim, npj Quantum Information **3**, 36 (2017).
[58] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, arXiv:1406.2661 [cs, stat] (2014), arXiv: 1406.2661, URL http://arxiv.org/abs/1406.2661.
[59] B. Uria, M.-A. Côté, K. Gregor, I. Murray, and H. Larochelle, Journal of Machine Learning Research **17**, 1 (2016).
[60] M. Germain, K. Gregor, I. Murray, and H. Larochelle, in *Proceedings of the 32nd International Conference on Machine Learning (ICML-15)* (2015), pp. 881–889.
[61] A. v. d. Oord, N. Kalchbrenner, and K. Kavukcuoglu, arXiv preprint arXiv:1601.06759 (2016).
[62] D. P. Kingma, S. Mohamed, D. J. Rezende, and M. Welling, in *Advances in Neural Information Processing Systems* (2014), pp. 3581–3589.
[63] R. Fergus, Y. Weiss, and A. Torralba, in *Advances in neural information processing systems* (2009), pp. 522–530.
[64] Y. Liu and K. Kirchhoff, in *INTERSPEECH* (2013), pp. 1840–1843.
[65] M. Shi and B. Zhang, Bioinformatics **27**, 3017 (2011).
[66] H. Chen and Z. Zhang, PloS one **8**, e62975 (2013).
[67] C. M. Bishop, *Pattern Recognition and Machine Learning* (Springer, New York, 2011), 1st ed., ISBN 978-0-387-31073-2.
[68] T. M. Cover and J. A. Thomas, *Elements of information theory*, Wiley series in telecommunications (Wiley, New York, 1991), ISBN 978-0-471-06259-2.
[69] M. D. Hoffman, D. M. Blei, C. Wang, and J. Paisley, The Journal of Machine Learning Research **14**, 1303 (2013).
[70] X. Chen, D. P. Kingma, T. Salimans, Y. Duan, P. Dhariwal, J. Schulman, I. Sutskever, and P. Abbeel, arXiv preprint arXiv:1611.02731 (2016).
[71] S. Zhao, J. Song, and S. Ermon, arXiv preprint arXiv:1706.02262 (2017).
[72] S. Yeung, A. Kannan, Y. Dauphin, and L. Fei-Fei, arXiv preprint arXiv:1706.03643 (2017).
[73] J. M. Tomczak and M. Welling, arXiv preprint arXiv:1705.07120 (2017).
[74] S. Zhao, J. Song, and S. Ermon, arXiv preprint arXiv:1702.08658 (2017).
[75] A. Mnih and K. Gregor, arXiv preprint arXiv:1402.0030 (2014).
[76] E. Jang, S. Gu, and B. Poole, arXiv preprint arXiv:1611.01144 (2016).
[77] L. Maaløe, M. Fraccaro, and O. Winther, arXiv preprint arXiv:1704.00637 (2017).
[78] A. Makhzani and B. Frey, arXiv preprint arXiv:1706.00531 (2017).
[79] J. Paisley, D. Blei, and M. Jordan, arXiv preprint arXiv:1206.6430 (2012).
[80] S. Gu, S. Levine, I. Sutskever, and A. Mnih, arXiv preprint arXiv:1511.05176 (2015).
[81] Y. Bengio, N. Léonard, and A. Courville, arXiv preprint arXiv:1308.3432 (2013).
[82] C. J. Maddison, A. Mnih, and Y. W. Teh, arXiv preprint arXiv:1611.00712 (2016).
[83] D. H. Ackley, G. E. Hinton, and T. J. Sejnowski, Cognitive science **9**, 147 (1985).
[84] S. Ioffe and C. Szegedy, in *International Conference on Machine Learning* (2015), pp. 448–456.
[85] D. Kingma and J. Ba, arXiv preprint arXiv:1412.6980 (2014).
[86] K. Hukushima and Y. Iba, in *AIP Conference Proceedings* (AIP, 2003), vol. 690, pp. 200–206.
[87] J. Machta, Physical Review E **82**, 026704 (2010).
[88] A. Mnih and D. Rezende, in *International Conference on Machine Learning* (2016), pp. 2188–2196.
[89] J. Bornschein and Y. Bengio, arXiv preprint arXiv:1406.2751 (2014).
[90] G. Tucker, A. Mnih, C. J. Maddison, J. Lawson, and J. Sohl-Dickstein, in *Advances in Neural Information Processing Systems* (2017), pp. 2624–2633.
[91] A. H. Khoshaman and M. H. Amin, arXiv preprint arXiv:1805.07349 (2018).
[92] E. Andriyash and M. H. Amin, arXiv preprint arXiv:1703.09277 (2017).
[93] H. Rieger and N. Kawashima, The European Physical Journal B-Condensed Matter and Complex Systems **9**, 233 (1999).

Appendix A: VAE with Guassian variables

In its simplest version, a VAE's prior and approximate posterior are a product of normal distributions (both with diagonal covariance matrix) chosen as follows:

$$\begin{aligned} p_{\boldsymbol{\theta}}(\boldsymbol{\zeta}) &= \mathcal{N}(\boldsymbol{\zeta};\mathbf{0},\mathbf{1}) \equiv \prod_{l=1}^{L}\mathcal{N}(\zeta_l;0,1) \\ q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x}) &= \mathcal{N}(\boldsymbol{\zeta};\boldsymbol{\mu},\boldsymbol{\sigma}) \equiv \prod_{l=1}^{L}\mathcal{N}(\zeta_l;\mu_l,\sigma_l)\,, \end{aligned}$$

where the prior is independent of the parameters $\boldsymbol{\theta}$ and the means and variances $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}^2$ are functions of the inputs $\mathbf{x}$ and of the parameters $\boldsymbol{\phi}$; the dependence on $\boldsymbol{\phi}$ is sometimes left implicit when the variable indices are shown. The mean and variance are usually the outputs of a deep neural network. The diagonal Guassians allow for an easy implementation of the reparameterization trick:

$$\rho_l \sim \mathcal{N}(\zeta_l;0,1),\ \zeta_l = \mu_l + \sigma_l\rho \ \Rightarrow\ \zeta_l \sim \mathcal{N}(\zeta_l;\mu_l,\sigma_l)\,.$$

The KL divergence is the sum of two simple Guassian integrals $D_{KL}(q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x})||p_{\boldsymbol{\theta}}(\boldsymbol{\zeta})) = -H(q_{\boldsymbol{\phi}}) + H(q_{\boldsymbol{\phi}},p_{\boldsymbol{\theta}})$:

$$\begin{aligned} H(q_{\boldsymbol{\phi}}) &= -\int q_{\boldsymbol{\phi}}(\boldsymbol{\zeta})\log q_{\boldsymbol{\phi}}(\boldsymbol{\zeta})d\boldsymbol{\zeta} = \\ &= \frac{1}{2}\sum_{l=1}^{L}(\log(2\pi)+1+\log(\sigma_l^2)) \\ H(q_{\boldsymbol{\phi}},p_{\boldsymbol{\theta}}) &= -\int q_{\boldsymbol{\phi}}(\boldsymbol{\zeta})\log p_{\boldsymbol{\theta}}(\boldsymbol{\zeta})d\boldsymbol{\zeta} = \\ &= -\frac{1}{2}\sum_{l=1}^{L}(\log(2\pi)+\mu_l^2+\sigma_l^2)\,. \end{aligned} \tag{A1}$$

The only term that requires the reparamaterization trick to obtain a low-variance estimate of the gradient is then the autoencoding term:

$$\mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x})}[\log p_{\boldsymbol{\theta}}(\mathbf{x}|\boldsymbol{\zeta})] \equiv \mathbb{E}_{\boldsymbol{\rho}}[\log p_{\boldsymbol{\theta}}(\mathbf{x}|\boldsymbol{\mu}+\boldsymbol{\sigma}\boldsymbol{\rho})]\,. \tag{A2}$$

Appendix B: DVAE with Bernoulli variables

The simplest DVAE can be implemented by assuming that the prior and the approximating posterior are both products of Bernoulli distributions

$$\begin{aligned} p_{\boldsymbol{\theta}}(z_l = 1) &= p_l \\ q_{\boldsymbol{\phi}}(z_l = 1|\mathbf{x}) &= q_l \,, \end{aligned}$$

where the Bernoulli probabilities q_l are functions of the inputs $\mathbf{x}$ and of the parameters $\boldsymbol{\phi}$ and are the outputs of a deep feed-forward network. We have already presented the following expression for the entropy in Sec. IV A:

$$\begin{aligned} H(q_{\boldsymbol{\phi}}) &\equiv -\mathbb{E}_{z\sim q_{\boldsymbol{\phi}}}[\log q_{\boldsymbol{\phi}}] = \\ &= -\sum_{l=1}^{L} \left(q_l \log q_l + (1-q_l)\log(1-q_l)\right) \,. \end{aligned} \tag{B1}$$

The cross-entropy can be derived similarly:

$$\begin{aligned} H(q_{\boldsymbol{\phi}}, p_{\boldsymbol{\theta}}) &\equiv -\mathbb{E}_{z\sim q_{\boldsymbol{\phi}}}[\log p_{\boldsymbol{\theta}}] = \\ &= -\sum_{l=1}^{L} \left(q_l \log p_l + (1-q_l)\log(1-p_l)\right) \,. \end{aligned} \tag{B2}$$

Similar to the fully Guassian case of the previous section, the only term that requires the reparameterization trick to obtain a low-variance estimate of the gradient is the autoencoding term as in Eq. 21 of the main text.

Appendix C: Hierarchical approximating posterior

Explain-away effects [6] introduce complicated dependencies in the approximating posterior $q_{\boldsymbol{\phi}}(\boldsymbol{\zeta}|\mathbf{x})$, which cannot be fully captured by products of independent distributions as we have considered so far. More powerful variational approximations of the posterior can be considered by including hierarchical structures. In the case of DVAEs, a hierarchical approximating posterior may be chosen as follows:

$$q_{\boldsymbol{\phi}}(z_l, \zeta_l|\mathbf{x}) = r(\zeta_l|z_l) q_{\boldsymbol{\phi}}(z_l|\zeta_{m<l}, \mathbf{x}) \,. \tag{C1}$$

A multivariate generalization of the reparameterization trick can be introduced by considering the conditional-marginal CDF defined as follows:

$$\mathrm{F}_l(\zeta_{m\leq l}) = \int_0^{\zeta_l} q_{\boldsymbol{\phi}}(\zeta_l'|\zeta_{m<l}, \mathbf{x}) d\zeta_l' \,, \tag{C2}$$

where in the expression above we assume the $\zeta_{m\neq l}$ are kept fixed. Thanks to the hierarchical structure of the approximating posterior, the $\mathrm{F}_l(\zeta_{m\leq l})$ functions are formally the same functions of ζ_l and q_l as in the case without hierarchies. The dependence of the functions $\mathrm{F}_l(\zeta_{m\leq l})$ on the continuous variables $\zeta_{m<l}$ is encoded in the functions $q_{\boldsymbol{\phi}}(\mathbf{z}, \boldsymbol{\zeta}|\mathbf{x})$:

$$\mathrm{F}_l(\zeta_{m\leq l}) = \mathrm{F}_l(\zeta_l, q_l(\zeta_{m<l})) \,. \tag{C3}$$

The reparameterization trick is again applied thanks to:

$$\rho_l \sim \mathcal{U}, \;\; \zeta_l = \mathrm{F}_l^{-1}(\rho_{m\leq l}) \quad \Rightarrow \quad \zeta_l \sim q_{\boldsymbol{\phi}}(\zeta_l|\mathbf{x}) \,.$$

The KL divergence is:

$$\begin{aligned} &D_{KL}(q_{\boldsymbol{\phi}}(\mathbf{z}, \boldsymbol{\zeta}|\mathbf{x})||p_{\boldsymbol{\theta}}(\mathbf{z}, \boldsymbol{\zeta})) = \\ &\quad = \sum_{l=1}^{L} D_{KL}(q_{\boldsymbol{\phi}}(z_l|\zeta_{m<l}, \mathbf{x}))||p_{\boldsymbol{\theta}}(z_l)) = \\ &\quad = \sum_{l=1}^{L} \mathbb{E}_{q_{\boldsymbol{\phi}}}[z_l \log q_l + (1-z_l)\log(1-q_l)] + \\ &\quad - \mathbb{E}_{q_{\boldsymbol{\phi}}}[\log p_{\boldsymbol{\theta}}(\mathbf{z})] \,. \end{aligned}$$

Notice that, due to the hierarchical structure of the approximating posterior, the expectations above cannot be performed analytically and must be statistically estimated with the use of the reparameterization trick.

Appendix D: Computing the derivatives

As shown in the previous section, the KL divergence generally includes a term that depends explicitly on the discrete variables z_l. When computing the gradients for back-propagation, we must account for the dependence of the discrete variables on the $\boldsymbol{\phi}$ parameters through the various hierarchical terms of the approximating posterior. Remembering that $z_l = \Theta(\rho_l + q_l - 1)$ and using the chain rule, we have:

$$\partial z_l = \partial_{q_l} z_l \partial_{\boldsymbol{\phi}} q_l = \delta(\rho_l + q_l - 1)\partial q_l \,. \tag{D1}$$

The gradient of the expectation over ρ of a generic function of z can then be calculated as follows:

$$\begin{aligned} \partial_{\boldsymbol{\phi}} \mathbb{E}_{\boldsymbol{\rho}}[f(\mathbf{z})] &= \mathbb{E}_{\boldsymbol{\rho}}[\partial_{\boldsymbol{\phi}} f(\mathbf{z})] = \sum_{l=1}^{L} \mathbb{E}_{\rho\sim\mathcal{U}}[\partial_{z_l} f(\mathbf{z}) \partial_{q_l} z_l \partial_{\boldsymbol{\phi}} q_l] = \\ &= \sum_{l=1}^{L} \mathbb{E}_{\rho_{k\neq l}}[\partial_{z_l} f(\mathbf{z}) \partial_{\boldsymbol{\phi}} q_l]_{\rho_l = 1-q_l \,\Rightarrow\, z_l = 0} = \\ &= \sum_{l=1}^{L} \mathbb{E}_{\boldsymbol{\rho}}[\partial_{z_l} f(\mathbf{z}) \frac{1-z_l}{1-q_l} \partial_{\boldsymbol{\phi}} q_l] = \\ &= \sum_{l=1}^{L} \mathbb{E}_{\boldsymbol{\rho}}[\partial_{z_l} f(\mathbf{z})(z_l - 1)\partial_{\boldsymbol{\phi}} \log(1-q_l)] \,, \end{aligned}$$

where, to go from the second to the third row, we have reinstated the expectation over ρ_l by noticing that q_l does not depend on ρ_l and that the condition $z_l = 0$ may be automatically enforced with the factor $1 - z_l$. The term $1 - q_l$ accounts for the fact that $z_l = 0$ with probability $1-q_l$. This term is necessary to account for the statistical dependence of z_l, and thus of $f(\mathbf{z})$, on variables $z_{m<l}$ that come before in the hierarchy. The equation derived above is useful to compute the derivatives of the positive phase

in the case of a hierarchical posterior and an RBM or a QRBM as priors:

$$\partial \mathbb{E}_{\boldsymbol{\rho}} \left[\log p_{\boldsymbol{\theta}}(\mathbf{z})\right] = \mathbb{E}_{\boldsymbol{\rho}}[\partial E_{\boldsymbol{\theta}}(\mathbf{z})] - \mathbb{E}_{p_\theta}[\partial E_{\boldsymbol{\theta}}(\mathbf{z})]\,, \tag{D2}$$

with

$$f(\mathbf{z}) = E_{\boldsymbol{\theta}}(\mathbf{z}) \quad \text{or} \quad \mathcal{H}_{\boldsymbol{\theta}}(\mathbf{z})\,. \tag{D3}$$

Appendix E: Population-annealed continuous-time quantum Monte Carlo

To sample from the quantum distribution Eq. 32 we use a continuous-time quantum Monte Carlo algorithm [93] together with a population annealing sampling heuristic [86, 87].

The CT-QMC algorithm is based on the representation of the quantum system with the Hamiltonian Eq. 32 in terms of a classical system with an additional dimension of size M called *imaginary time*. The classical configuration $\mathbf{z}$ is replaced with M configurations $\mathbf{z}^a, a = 1, \ldots, M$ that are coupled to each other in a periodic manner. The quantum partition function $Z_{\boldsymbol{\theta}} = \mathrm{Tr}[e^{-\mathcal{H}_{\boldsymbol{\theta}}}]$ can be written as:

$$Z_{\boldsymbol{\theta}} \simeq \sum_{\{\mathbf{z}^a\}} \exp\left\{\log \frac{\Gamma}{M} \sum_{i,a} \frac{1 - z_i^a z_i^{a+1}}{2} - \frac{1}{M}\sum_{a=1}^{M} \mathcal{H}_0(\mathbf{z}^a)\right\}. \tag{E1}$$

Here $\mathcal{H}_0$ is the classical energy $H_0(\mathbf{z}) = \sum_l z_l h_l + \sum_{l<m} W_{lm} z_l z_m$ and periodicity along imaginary time implies $\mathbf{z}^{M+1} \equiv \mathbf{z}^1$.

CT-QMC defines a Metropolis-type transition operator acting on extended configurations $T_{\boldsymbol{\theta}} : \mathbf{z}^a \to \mathbf{z}^{a\prime}$. We use cluster updates [93] where clusters may grow only along the imaginary time direction. These updates satisfy detailed balance conditions for the distribution

$$\begin{aligned} p_{\boldsymbol{\theta}}(\mathbf{z}^a) &= e^{-E_q(\mathbf{z}^a) - E_{cl}(\mathbf{z}^a)}/Z_{\boldsymbol{\theta}} \\ E_q(\mathbf{z}^a) &= -\log \frac{\Gamma}{M} \sum_{i,a} \frac{1 - z_i^a z_i^{a+1}}{2} \\ E_{cl}(\mathbf{z}^a) &= \frac{1}{M}\sum_{a=1}^{M} H_0(\mathbf{z}^a). \end{aligned} \tag{E2}$$

Equilibrium samples from Eq. E2 allow us to compute the gradient of the bound on the log-likelihood in Eq. 36 as

$$\mathbb{E}_{p_\theta(\mathbf{z})}[\partial \mathcal{H}_{\boldsymbol{\theta}}(\mathbf{z})] = \mathbb{E}_{p_\theta(\mathbf{z}^a)}[\partial \mathcal{H}_{\boldsymbol{\theta}}(\mathbf{z}^1)]\,. \tag{E3}$$

To obtain approximate samples from Eq. E2, we use PA, which also gives an estimate of the quantum partition function [86, 87]. We choose a linear schedule in the space of parameters $\boldsymbol{\theta}_t = t\boldsymbol{\theta}, t \in [0, 1]$ and anneal an ensemble of N particles $\mathbf{z}_n^a, n = 1, \ldots, N$ with periodic resampling.

Finally, we must evaluate the quantum cross-entropy Eq. 33, which involves computing probabilities of classical configuration $\bar{\mathbf{z}}$ under the quantum distribution $p_{\boldsymbol{\theta}}(\bar{\mathbf{z}}) \equiv \mathrm{Tr}[\Lambda_{\bar{\mathbf{z}}} e^{-\mathcal{H}_{\boldsymbol{\theta}}}]$. This is done by noticing that

$$\begin{aligned} &\mathrm{Tr}[\Lambda_{\bar{\mathbf{z}}} e^{-\mathcal{H}_{\boldsymbol{\theta}}}] = \langle \bar{\mathbf{z}} | e^{-\mathcal{H}_{\boldsymbol{\theta}}} | \bar{\mathbf{z}} \rangle \simeq \\ &\simeq \sum_{\{\mathbf{z}^a, a=2..M\}} \exp\left\{-E_q(\mathbf{z}^a) - E_{cl}(\mathbf{z}^a) - E_{\text{boundary}}(\mathbf{z}^a)\right\}, \\ &E_{\text{boundary}}(\mathbf{z}^a) = -\log \frac{\Gamma}{M} \sum_i \frac{2 - \bar{z}_i z_i^2 - \bar{z}_i z_i^M}{2}. \end{aligned} \tag{E4}$$

Thus, to obtain $p_{\boldsymbol{\theta}}(\bar{\mathbf{z}})$, we must compute the partition function $\langle \bar{\mathbf{z}} | e^{-\mathcal{H}_{\boldsymbol{\theta}}} | \bar{\mathbf{z}} \rangle$ of a "clamped" system, where the first slice of imaginary time is fixed $\mathbf{z}^1 \equiv \bar{\mathbf{z}}$ and we integrate out the rest of the slices taking into account the external field acting on slices 2 and M.